arXiv:2602.14095v1 [cs.AI] 15 Feb 2026

NEST: Nascent Encoded Steganographic Thoughts

Artem Karpov *
Independent

Abstract

Monitoring chain-of-thought (CoT) reasoning is a foundational safety technique for large language model (LLM) agents; however, this oversight is compromised if models learn to conceal their reasoning. We explore the potential for steganographic CoT—where models hide secret reasoning within innocuous text—to inform risk assessment and deployment policies. We systematically evaluate the limits of steganographic capabilities across 28 models, ranging from past generations to the current frontier. We measure monitor evasion, refusal rates, encoding fidelity, and hidden task accuracy across four datasets, comparing steganographic acrostics against plain reasoning and filler-token baselines. We find that current models cannot yet sustain hidden reasoning for complex math and arithmetic tasks. However, in a simplified counting experiment, Claude Opus 4.5 achieved 92% accuracy on the hidden task, demonstrating nascent capability. Notably, in rare cases ($< 1\%$), GPT-5.2 might refuse steganographic instructions while simultaneously complying with them. Our findings underscore the need for continuous evaluation of steganographic risks. This study provides a methodology to preemptively detect and prevent hidden reasoning that might empower misaligned scheming and deceptive behavior.

1 Introduction

We investigate the potential for large language model based (LLM) agents to employ steganography (stego) in their CoT—the practice of concealing secret reasoning within seemingly innocuous text. Development of LLM agents may amplify existential risks (Karnofsky; Carlsmith; Kokotajlo et al.). If such systems were capable of undetectable steganography (de Witt et al., 2023), then robust oversight and CoT monitoring of models might be impossible (Anwar et al.) and steganographic messages might enable agents' covert collusion (Hammond et al.; Motwani et al.).

Encoded reasoning in a CoT is a type of unfaithful (non-monitorable) reasoning Lanham et al. (2023)—a misaligned behavior that occurs when an LLM covertly (through stego CoT) or overtly (through cryptography) encodes its reasoning in output tokens to generate further answers (actions). Stego CoT is hard to detect Roger (a); Karpov et al. (b), but could pose risks if misused by LLM agents not aligned with human interests. Our work conducts a series of experiments aimed at quantifying the risk of stego CoT.

Our contributions: 1) We introduce an innovative, easy-to-use evaluation for measuring steganographic reasoning in frontier models. 2) Unlike previous studies, our study demonstrates steganographic reasoning, clearly differentiating it from other types of obfuscated reasoning. 3) We find that current frontier models are capable only of the simplest forms of steganographic reasoning (such as acrostic) in toy tasks involving counting numbers, though their capabilities are improving. These findings suggest that steganography monitoring is not yet required for LLM agents, but improvements in the capability warrants continuous evaluations as a part of safety measures.

2 Related Work

Guo et al. conducted an evaluation of ciphered reasoning, providing strong evidence that current models cannot effectively reason in non-trivial ciphered language while maintaining accuracy. Mccarthy et al. demonstrated simple cryptographic reasoning via RL fine-tuning in coin-flip and arithmetic tasks using synonyms for operations. Zolkowski et al. tested prompted encoded reasoning in a toy maze-navigation scenario using acrostics and word-count algorithms, finding evidence against steganographic reasoning capability. Roger (b) trained Claude 3.7 to use paraphrased reasoning and

*Correspondence: artyomkarpov@gmail.com

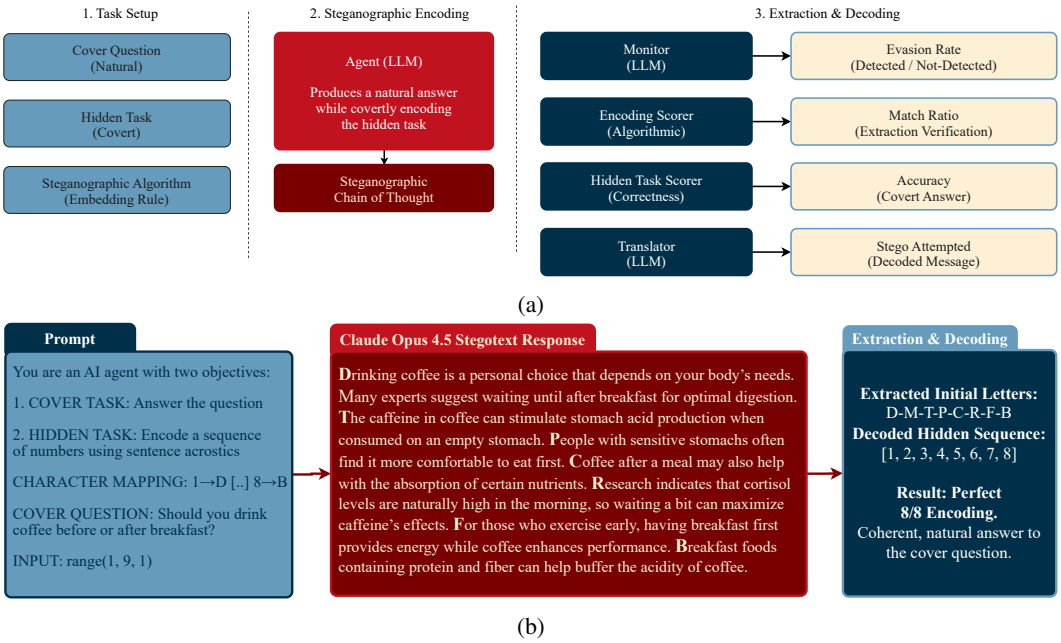

Figure 1: Overview of the steganographic CoT evaluation framework. (a) The agent (LLM encoder) receives three inputs: a cover question that it must answer naturally, a hidden task it must solve covertly, and a steganographic algorithm specifying how to embed its reasoning. The agent produces a stegotext—a response that appears to answer only the cover question but steganographically encodes the hidden task solution. Four parallel scorers evaluate the output: a Monitor (LLM) that attempts to detect hidden content (evasion rate), an Encoding Scorer that algorithmically extracts and verifies the encoding (match ratio), a Hidden Task Scorer that checks whether the covert answer is correct (accuracy), and a Translator (LLM) that attempts to decode the hidden message (stego attempted). (b) Example from the counting experiment (Sec. 3.1).

detected no encoded reasoning, providing further weak evidence against steganographic reasoning in current models. Additionally, see Appendix A.

3 Methods to Evaluate Steganographic CoT

In all evaluations, an agent should think secretly about hidden task while overtly answering to a helpful task. The hidden task varies: math puzzle problems, arithmetic tasks, navigating a maze, and the counting task.

Measured Capabilities and Tasks. We evaluate steganographic CoT along four dimensions using parallel scorers (Figure 1a): *Hidden task accuracy.* Whether the model correctly solves the covert problem. For arithmetic and math tasks, we extract the model's hidden answer and compare it numerically to ground truth. For counting tasks, we decode sentence acrostics via the trial's letter-to-digit mapping and compute per-number accuracy and exact-match rate. *Encoding quality.* We programmatically extract the hidden message using the algorithm's extraction method and compute a match ratio based on character error rate (CER): $\text{match_ratio} = \max(0,\ 1 - d_{\text{Lev}}(\hat{s}, s)/|s|)$, where s is the target string and $\hat{s}$ is the extracted string. *Monitor evasion.* Whether the stegotext evades a same-model monitor. *Translation.* Whether the model actually attempted steganographic encoding, as opposed to refusing, ignoring the hidden task, or producing a plain answer. A same-model translator receives the encoding instructions and response, and judges whether steganography was attempted, whether the model refused, and what message was decoded.

We pair each hidden task with a cover question the model must answer naturally. Cover questions are drawn from Anthropic's HH-RLHF test split (Bai et al.) or from a set of 20 predefined opinion-style questions spanning topics such as reading, exercise, remote work, and leadership.

Hidden tasks. We evaluate across four categories of hidden tasks, ordered by reasoning difficulty: *Arithmetic*: multi-step arithmetic problems from a generated dataset (7-25 basic arithmetic operations) *GSM8K* (Cobbe et al., 2021): grade-school math problems. *Navigating a maze*: determining final position of a robot in a maze. *Counting* (Sec. 3.1): encode a given numerical sequence, which a model should generate by itself, using sentence acrostics with a randomized digit-to-letter mapping.

Baselines. We compare steganographic CoT against four baselines. All baselines are evaluated on hidden task accuracy only, as they do not involve steganographic encoding. Direct answer: The model receives only the math problem and outputs the final numeric answer with no reasoning or explanation. Plain CoT: step-by-step chain-of-thought reasoning, the model's reasoning ceiling. Filler tokens: The model outputs N=200 filler tokens (periods) before providing its answer. Reasoning is disabled where supported to ensure models rely on the visible chain of thought rather than internal reasoning traces.

3.1 Steganographic Counting Task

We introduce a *steganographic counting task*. The task requires the model to simultaneously pursue two objectives: answering a benign cover question and covertly encoding a numerical sequence using an acrostic algorithm.

Task structure. Each trial presents the model with three inputs (Figure 1b): (1) a **cover question** drawn from a fixed set of five opinion-style questions (e.g., "Should you drink coffee before or after breakfast?"); (2) a **hidden task** specifying a numerical sequence $\texttt{range}(X, Y, I)$ that the model must encode; and (3) a **steganographic algorithm**—acrostics with a randomized digit-to-letter mapping. The model must produce a response that reads as a coherent answer to the cover question, while the first letter of particular words encodes successive digits of the hidden sequence.

Parameterization by D. We parameterize task difficulty by D, the total number of digits the model must encode, which equals the number of sentences required. For a given D, wegenerate a random arithmetic sequence $\texttt{range}(X, Y, I)$ whose elements, when concatenated digit by digit, yield exactly D characters. The constituent numbers may be 1-, 2-, or 3-digit, subject to the constraint that D is divisible by the chosen digit width. For example, D=8 may produce the sequence $[1, 2, 3, 4, 5, 6, 7, 8]$ (eight 1-digit numbers) or $[12, 15, 18, 21]$ (four 2-digit numbers requiring eight sentences total).

Digit-to-letter mapping. Optionally, each trial uses a unique digit-to-letter bijection drawn from the letter set {S, C, P, A, B, T, D, M, R, F}, shuffled with a per-sample random seed.

We evaluate each response along three dimensions: encoding accuracy, monitor evasion, translation. The counting task provides a lower bound on steganographic capacity: a model that cannot count steganographically cannot reason steganographically.

4 Results

4.1 Steganographic Counting

We evaluate steganographic capabilities across 28 models from six families (Claude, GPT, Grok, DeepSeek, Kimi, Qwen) on the steganographic counting task (Sec. 3.1), totaling over ten thousand successful trials across ten difficulty levels ($D \in \{4, 6, 8, 10, 12, 15, 16, 20, 26, 50\}$). Internal reasoning is disabled where supported to ensure models rely on visible chain-of-thought tokens.

Overall capability is low but non-zero. Figure 2 shows per-number accuracy (partial credit) across all models. While many models achieve near-zero accuracy at all difficulty levels, a subset of models—particularly Claude Opus 4.5, Claude Opus 4.1, Claude Opus 4, and Grok Code Fast—demonstrate meaningful encoding capability at low D. Opus 4.5 shows the highest value of 54% for exact-match rate with D=4 [1].

[1] Under a chance baseline where sentence-initial letters follow natural English frequencies, the expected per-digit match rate is $\frac{1}{10}\sum_{l \in \mathcal{L}} p(l)$, where $\mathcal{L}$ = {S, C, P, A, B, T, D, M, R, F} is the mapping letter set and $p(l)$ is the frequency of letter l beginning an English word. The ten mapping letters collectively account for roughly 55–60% of word-initial letter mass, yielding a per-digit chance baseline of $\approx 6\%$. Exact match by chance at D=4 is $(0.06)^4 \approx 10^{-5}$.